Introduction to Computer Systems

Lecture 12: Database Transactions

Instructors: Wei Xu, Mingyu Gao
Teaching Assistants: Weitao Wan, Tingrui Zhang, Ruitao Liu, Shuo Chen

Contents

1 Introduction **3**

2 From File Systems to Database Systems **3**
2.1 Why Study Databases in a Systems Course? . . . 3
2.2 Structured Data, Data Models, and Schemas . . . 4
2.3 What a DBMS Provides . . . 5

3 Queries, Operators, and Query Plans **5**
3.1 Declarative Queries . . . 5
3.2 Selection, Projection, and Join . . . 6
3.3 Why Query Plans Matter . . . 7

4 Transactions and ACID **7**
4.1 Transactions as Atomic Sequences of Storage Actions . . . 7
4.2 The ACID Properties . . . 8

5 Atomic Writes to Persistent Storage **10**
5.1 Why Disk Updates Are Harder Than Memory Updates . . . 10
5.2 The Concept of a Log . . . 10
5.3 Creating a File as a Logged Transaction . . . 12
5.4 Recovery Cases . . . 13
5.5 Journaling File Systems . . . 14

6 Isolation and Concurrent Transaction Execution **16**
6.1 From Multiprogramming to Transactions . . . 16
6.2 Database Locks Are Expensive . . . 16

7 Transaction Scheduling and Anomalies **17**
7.1 The Goal of Transaction Scheduling . . . 17
7.2 Why Anomalies Always Involve Writes . . . 19
7.3 Write-Write Conflict . . . 19
7.4 Read-Write Conflict and Unrepeatable Reads . . . 19
7.5 Write-Read Conflict and Dirty Reads . . . 20

8 Serializability and Conflict Serializability **20**
8.1 Equivalent and Serializable Schedules . . . 20
8.2 Conflicting Operations . . . 20
8.3 Dependency Graphs . . . 22
8.4 Examples and Limits . . . 24

9 Locks and Two-Phase Locking **26**
9.1 Shared and Exclusive Locks 26
9.2 Why Immediate Unlocking Is Not Enough 26
9.3 Two-Phase Locking 27
9.4 Why 2PL Guarantees Conflict Serializability 27
9.5 Strict 2PL and Optimistic Concurrency Control 28

10 Putting the Lecture Together **29**

11 Summary **29**

12 Quizzes & Exercises **30**

1 Introduction

This lecture note is organized around the technical scope of Lecture 12: database systems, structured data, query execution, transactions, atomic disk updates, serializability, conflict serializability, and locking. The spoken material around this lecture crosses recording boundaries: the beginning of the transaction topic appears after networking and RPC, while the later discussion of serializability and two-phase locking continues before distributed systems. The note therefore follows the slide boundary rather than the recording boundary. Distributed file systems, NFS, AFS, Coda, and broader distributed-systems material are reserved for the following lecture.

The central idea of this lecture is that a database transaction generalizes several mechanisms already seen in operating systems. A single machine OS tries to make memory access, file-system metadata updates, synchronization, and process isolation behave predictably. A database management system (DBMS) must solve analogous problems, but with harder constraints: persistent data on disks or SSDs, expensive I/O, long-running operations, many simultaneous users, crashes, and application-level correctness rules such as "money is not created or destroyed" or "a course enrollment must refer to an existing course."

Definition

A **transaction** is a sequence of database actions, usually reads and writes, that the system treats as one logical operation. A correct transaction takes the database from one consistent state to another, and the DBMS provides mechanisms to make the sequence appear atomic, isolated, and durable.

A convenient slogan is:

OS synchronization protects shared memory; transactions protect persistent shared data.

The slogan is useful but incomplete. Database locks are not merely a few CPU instructions around a short critical section. They may involve lock tables, deadlock handling, recovery logs, disk writes, query plans, indexes, buffer managers, and interactions with transactions that run for seconds, minutes, or much longer. Consequently, the design space is richer than simply "lock everything until the operation finishes."

2 From File Systems to Database Systems

2.1 Why Study Databases in a Systems Course?

A DBMS is one of the most important classes of systems software. It stores and manages the persistent state behind banking systems, e-commerce platforms, university records, configuration databases, cloud services, and many operating-system services. From the systems point of view, a DBMS is an advanced storage manager that provides stronger abstractions than a normal file system:

- structured data rather than untyped byte streams;
- queries rather than manual scanning loops;
- indexes and query plans rather than only file offsets;
- transactions rather than ad hoc sequences of writes;
- concurrency control rather than only process-level locking;
- recovery after crashes rather than only best-effort repair.

The contrast with file systems is important. A file system typically stores files as bytes plus metadata such as inode numbers, directory entries, permissions, and timestamps. A DBMS understands higher-level concepts such as tables, fields, types, keys, constraints, and relationships. This additional structure lets it provide richer correctness and performance guarantees.

2.2 Structured Data, Data Models, and Schemas

A *data model* describes what kinds of entities exist and how they relate. A *schema* is a concrete instance of a data model: it specifies the fields, their types, and the organization of the database. The most widely used model is the relational model, in which a relation is a table with rows and columns.

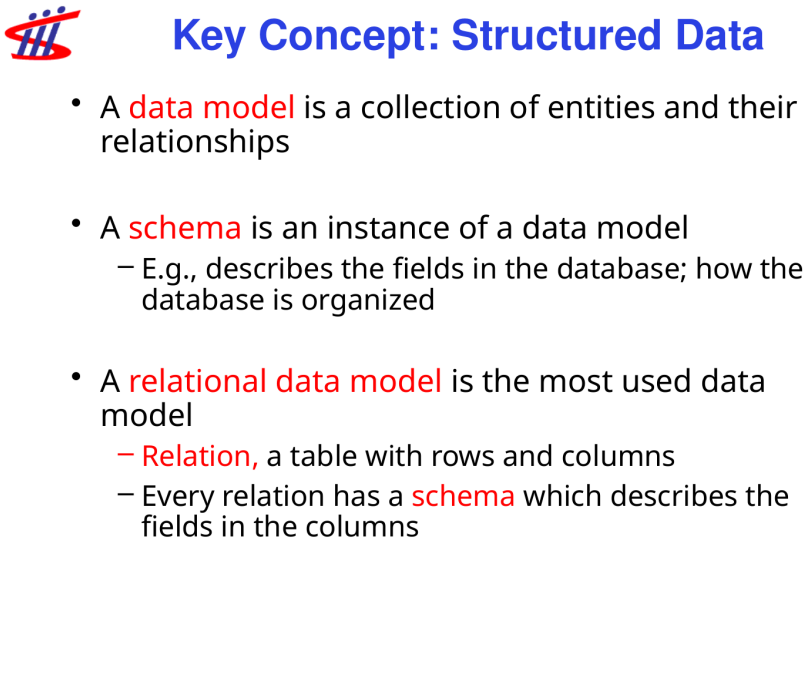


Figure 1: Structured data gives the DBMS semantic information that ordinary files do not provide. The schema defines fields and organization; the relational model represents data as tables with rows and columns.

Consider a course database with three relations:

```
CREATE TABLE Students (
    sid   VARCHAR(32) PRIMARY KEY,
    name  VARCHAR(128),
    email VARCHAR(128),
    age   INTEGER,
    gpa   REAL
);

CREATE TABLE Courses (
    cid     VARCHAR(32) PRIMARY KEY,
    cname   VARCHAR(128),
    credits INTEGER
);

CREATE TABLE Enrolled (
    sid   VARCHAR(32),
    cid   VARCHAR(32),
```

```
    grade VARCHAR(8),
    FOREIGN KEY (sid) REFERENCES Students(sid),
    FOREIGN KEY (cid) REFERENCES Courses(cid)
);
```

Listing 1: A simple relational schema for courses and enrollment.

The foreign keys express relationships without using memory pointers. A pointer is an address in a particular process at a particular moment. It is meaningless after a restart, on a different machine, or after the object is moved. A database instead uses stable values such as student IDs and course IDs. This is a fundamental distinction between persistent structured data and in-memory object graphs.



Example: Course Database

- Conceptual schema:

  Students(sid*: string*, name*: string*, email*: string*, age*: integer*, gpa*:real*)

  Courses(cid*: string*, cname*:string*, credits*:integer*)

  Enrolled(sid*:string*, cid*:string*, grade*:string*)

  FOREIGN KEY sid REFERENCES Students

  FOREIGN KEY cid REFERENCES Courses

7

Figure 2: The course database schema uses three relations. The Enrolled table stores relationships between students and courses through foreign keys rather than memory pointers.

2.3 What a DBMS Provides

A database management system stores, manages, and facilitates access to databases. It typically provides a Data Definition Language (DDL), a Data Manipulation Language (DML), and guarantees about durability, concurrency, and semantics. DDL defines schemas; DML retrieves, analyzes, inserts, deletes, and updates data.

The DBMS is therefore both a programming interface and a storage system. It must translate high-level requests into efficient low-level disk and memory operations while preserving correctness across crashes and concurrent access.

3 Queries, Operators, and Query Plans

3.1 Declarative Queries

SQL is a declarative language. Instead of saying how to loop over records, it says what result is desired. For example:



What is a Database System?

- A **Database Management System (DBMS)** is a software system designed to store, manage, and facilitate access to databases.
- A DBMS provides:
  - Data Definition Language (DDL)
    - » Define relations, schema
  - Data Manipulation Language (DML)
    - » Queries – to retrieve, analyze and modify data.
  - Guarantees about durability, concurrency, semantics, etc

8

Figure 3: A DBMS provides language interfaces for defining and manipulating structured data, plus system guarantees such as durability and concurrency control.

```sql
SELECT sid, name, gpa
FROM Students S
WHERE S.gpa > 3;
```

Listing 2: Selecting students with GPA greater than 3.0.

An imperative C or Python program might open a file, read every student record, check the GPA, and copy selected fields into an output buffer. SQL leaves these execution choices to the DBMS. This is not merely a convenience feature; it is an optimization opportunity. The database can choose a plan based on indexes, table sizes, available memory, storage layout, and statistics.

3.2 Selection, Projection, and Join

Three relational operators are especially important:

- **Selection** filters rows according to a predicate, such as `gpa` > 3.
- **Projection** selects columns, such as `sid`, `name`, and `gpa`.
- **Join** combines rows from different tables according to a relationship, often equality of keys.

SQL terminology can be slightly confusing: the `SELECT` clause performs projection, while the `WHERE` clause often performs selection. The names come from the language syntax rather than directly from relational algebra.

A more complex query asks how many distinct students take a four-credit course:

```sql
SELECT COUNT(DISTINCT E.sid)
FROM Enrolled E, Courses C
WHERE E.cid = C.cid
  AND C.credits = 4;
```

Listing 3: Counting distinct students enrolled in four-credit courses.

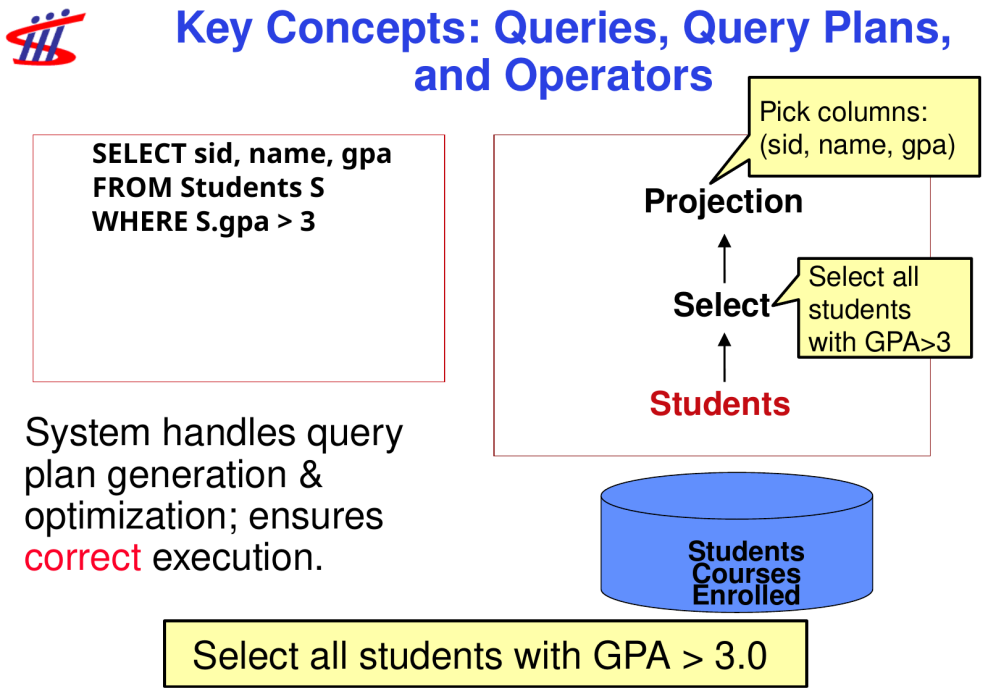


Figure 4: A simple query plan can be viewed as a pipeline of relational operators: first select rows satisfying the GPA predicate, then project the desired columns.

The logical plan joins `Enrolled` with `Courses`, filters courses with four credits, and counts distinct student IDs. A naive join compares every row of one table with every row of the other, giving a cost proportional to the product of table sizes. Real systems use indexes, hashing, sorting, and join-order optimization.

3.3 Why Query Plans Matter

Because SQL is declarative, the same query can have many correct physical implementations. If a table is stored by rows, filtering rows first may be efficient. If it is stored by columns and contains large fields, projecting only the needed columns first may reduce I/O. If the predicate is highly selective, pushing selection down before a join can reduce the input size dramatically. If there is an index on `credits`, the DBMS may avoid scanning all courses.

Deep Dive

Query optimization is one reason databases are a systems field rather than only an application library. The optimizer must reason about algorithms, data structures, statistics, memory hierarchy, disk access, and concurrency. An apparently simple SQL statement may become a tree of physical operators with buffer-management decisions at every stage.

4 Transactions and ACID

4.1 Transactions as Atomic Sequences of Storage Actions

A transaction is an atomic sequence of database actions, typically reads and writes. It takes the database from one consistent state to another. The word *consistent* here refers to application semantics: balances obey financial rules, dates are valid, foreign keys refer to existing records,

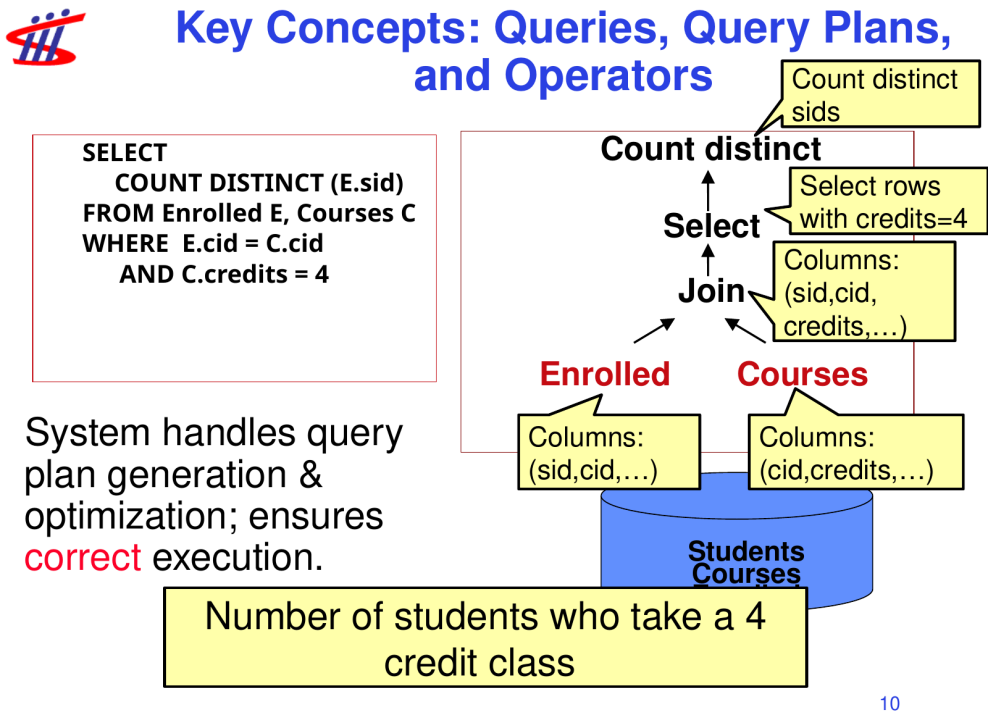


Figure 5: A join query combines enrollment records with course records, filters the resulting rows by credit count, and then counts distinct student identifiers.

and so on.

The database cannot infer every semantic rule automatically. It can enforce declared integrity constraints such as nonnegative balances, valid foreign keys, uniqueness, or not-null fields, but the transaction writer is responsible for preserving the larger application invariant. For example, a transfer between two accounts should preserve total money except for explicitly modeled fees.

4.2 The ACID Properties

The classical transaction properties are ACID:

Atomicity All actions in the transaction happen, or none happen. A crash or abort must not leave half of the transaction visible as a committed result.

Consistency If the database begins in a consistent state and the transaction is correctly written, the database ends in a consistent state. The DBMS can assist through integrity constraints, but application semantics matter.

Isolation Concurrent transactions should not interfere in ways that violate the illusion that each transaction ran alone or in some serial order.

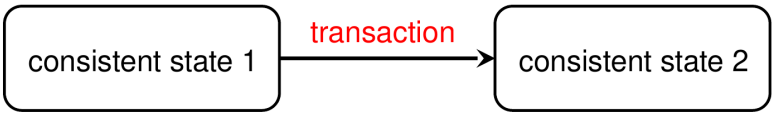
Durability Once a transaction commits, its effects persist despite crashes.

A simple bank transfer illustrates why ACID properties are inseparable. The transfer should subtract money from Alice, add money to Bob, and update any branch-level summary tables consistently. If the system crashes after subtracting from Alice but before adding to Bob, atomicity is violated. If a concurrent transaction reads Alice's new balance but Bob's old balance, isolation is violated. If the transaction commits but the data is lost after reboot, durability is violated.



Key concept: Transaction

- An atomic sequence of database actions (reads/writes)
- Takes DB from one consistent state to another



12

Figure 6: A transaction is a logical transition from one consistent database state to another. The DBMS treats the internal sequence of reads and writes as a unit.



The ACID properties of Transactions

- Atomicity: all actions in the transaction happen, or none happen
- Consistency: transactions maintain data integrity, e.g.,
  - Balance cannot be negative
  - Cannot reschedule meeting on February 30
- Isolation: execution of one transaction is isolated from that of all others; no problems from concurrency
- Durability: if a transaction commits, its effects persist despite crashes

14

Figure 7: ACID summarizes the contract that transaction systems try to provide: all-or-nothing execution, semantic preservation, isolation from concurrent transactions, and persistence after commit.

```
BEGIN;
UPDATE accounts
   SET balance = balance - 100.00
 WHERE name = 'Alice';

UPDATE branches
   SET balance = balance - 100.00
 WHERE name = (SELECT branch_name
                 FROM accounts
                WHERE name = 'Alice');

UPDATE accounts
   SET balance = balance + 100.00
 WHERE name = 'Bob';

UPDATE branches
   SET balance = balance + 100.00
 WHERE name = (SELECT branch_name
                 FROM accounts
                WHERE name = 'Bob');
COMMIT;
```

Listing 4: A bank transfer grouped as one transaction.

5 Atomic Writes to Persistent Storage

5.1 Why Disk Updates Are Harder Than Memory Updates

In memory, a load or store of a properly aligned machine word is often treated as atomic by the hardware and programming model. Persistent storage is different. Even writing a single disk block may not be atomic under power failure. A crash can occur after some blocks are written and before others, or during a block overwrite. A multi-block update, such as creating a file or transferring money between accounts, is therefore not naturally atomic.

The file-system example is concrete. Creating a file may require allocating a data block, writing the data, allocating an inode, writing the inode, updating free-space bitmaps, inserting a directory entry, and updating timestamps. If a crash occurs in the middle, the disk may contain a directory entry pointing to an unwritten inode, an allocated block not referenced by any inode, an inode not reachable from any directory, or timestamps inconsistent with directory contents.

Traditional recovery tools such as `fsck` or `scandisk` scan metadata after a crash and try to rebuild consistency. This can take time proportional to disk size and may not recover the user's intended state. The key lesson is that checking after the fact is expensive and incomplete. A better design records enough information before modifying the main data structures so recovery is deterministic.

5.2 The Concept of a Log

A log converts a sequence of physical updates into one logical action by exploiting an operation that can be made relatively simple and atomic: appending or writing a basic log record. The log records the system's intention before the main data is modified. A commit record then seals the transaction.

The principle is often called write-ahead logging (WAL): before a change is installed in its home location, enough information about the change must be safely written to the log. If the machine crashes, recovery scans the log. A transaction with no commit record is discarded. A transaction with a commit record is redone until the home locations reflect the logged update.



Operations on persistent storage are not atomic !

- Recall: in-memory operations
  - Load / Store on a single word is atomic
  - Basic operations are atomic
- Atomicity on persistent storage
  - Even writing a single block is not atomic
  - Many operations involves many reads/writes
  - If broken, cannot recovery from a single reboot
- E.g. File system update
  - Some operations in progress may complete
  - Some operations in progress may be lost
  - Overwrite of a block may only partially complete

17

Figure 8: Persistent storage operations are not automatically atomic. Crashes can leave some writes completed, some lost, and some partially completed.



File Systems: Create a File

Normal operation:

- Allocate data block
- Write data block
- Allocate inode
- Write inode block
- Update bitmap of free blocks and inodes
- Update directory with file name → inode number
- Update modify time for directory

Recovery:

- Scan inode table
- If any unlinked files (not in any directory), delete or put in lost & found dir
- Compare free block bitmap against inode trees
- Scan directories for missing update/access times

Time proportional to disk size

19

Figure 9: Creating a file is a sequence of metadata and data operations. Without transactional protection, recovery may require scanning large parts of the disk to repair inconsistencies.

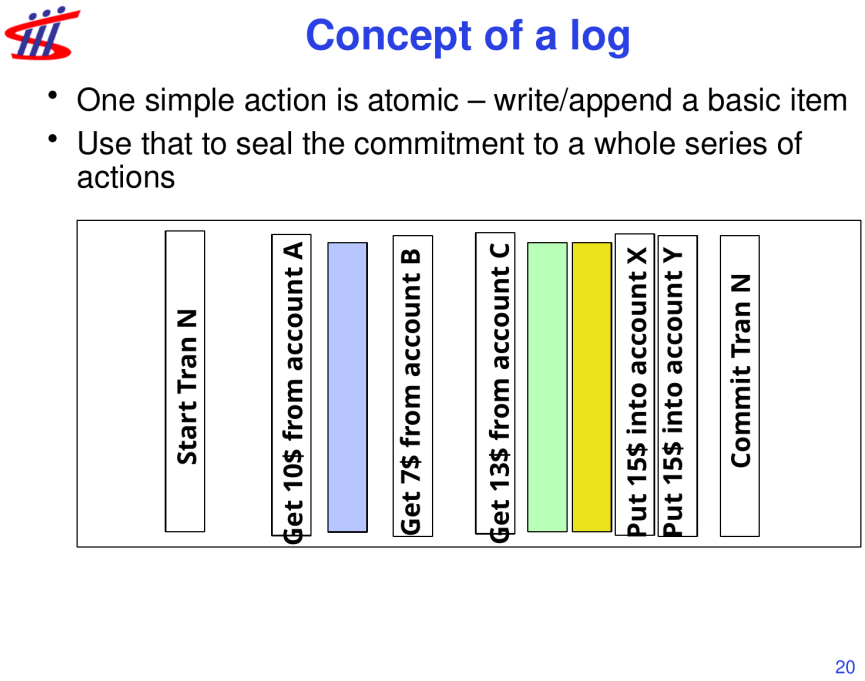


Figure 10: A log records the start and commit of a transaction. The commit record marks the point at which the system must preserve the transaction's effects.

Definition

A **redo log** records the new values that must eventually appear in the database or file-system structures. During recovery, committed log records are replayed. Replaying the same committed update multiple times is safe when the replay action is idempotent, such as writing the same block contents to the same location.

5.3 Creating a File as a Logged Transaction

A logged file creation first finds the free data blocks, free inode entry, and directory insertion point. Instead of immediately updating the free-space map, inode table, and directory, the file system writes intended modifications to a nonvolatile log. After all log records are present, it writes the commit record. Only then does the transaction count as committed.

Once committed, the system can apply the changes to the real file-system structures. It may do so immediately or later. Until the log is checkpointed or truncated, accesses may need to consult the log to see the latest value.

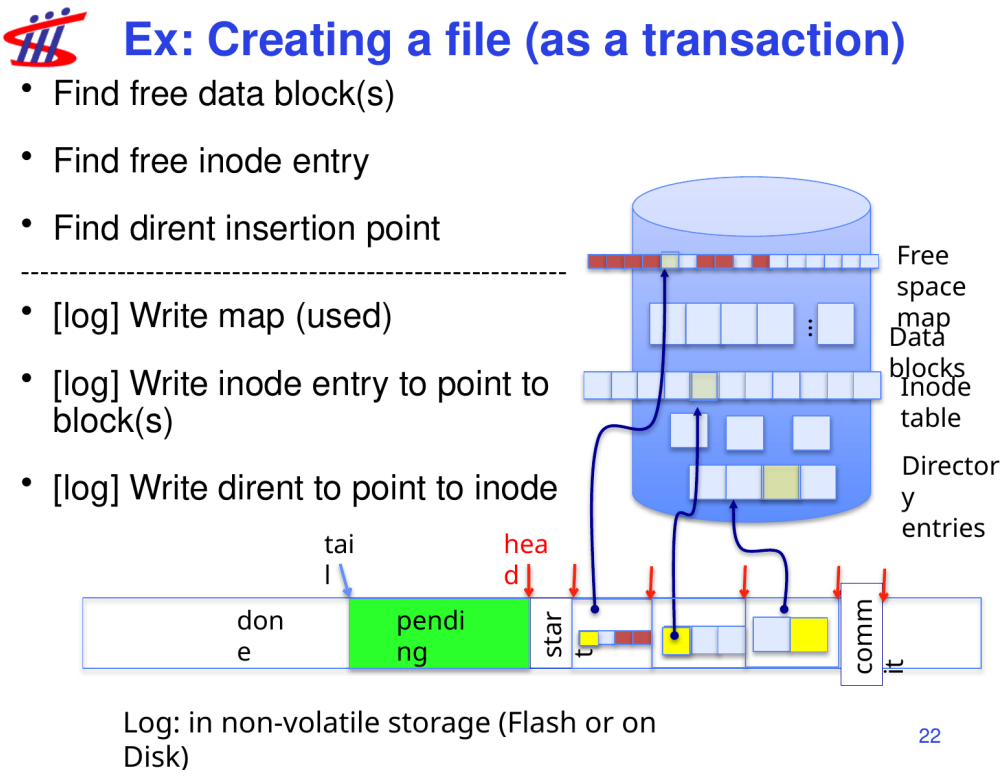


Figure 11: Creating a file as a transaction: intended changes to the free-space map, inode table, and directory are logged before they are installed in their home locations.

Algorithm 1: Redo Logging for a Storage Transaction

Input: Transaction identifier T; ordered list of home-location updates $U = [(a_1, v_1), \ldots, (a_k, v_k)]$

Output: Committed durable update or abort before commit

```
Append START(T) to the log and force it to nonvolatile storage;
foreach (a_i, v_i) ∈ U do
    Append UPDATE(T, a_i, v_i) to the log;
Force all update log records for T to nonvolatile storage;
Append COMMIT(T) to the log and force it to nonvolatile storage;
foreach (a_i, v_i) ∈ U do
    Write value v_i to home location a_i;
return committed;
```

The force operations are crucial. If the commit record reaches stable storage before the update records do, recovery might see a committed transaction without enough information to redo it. Real storage systems use checksums, ordering barriers, flush commands, group commit, and careful block layout to implement this safely and efficiently.

5.4 Recovery Cases

There are two essential crash cases.

First, the system may crash during logging before the commit record is durable. Recovery scans the log, sees a transaction start but no matching commit, and discards those log entries. The main disk state is treated as unchanged by that transaction.

Second, the system may crash after commit but before all home locations are updated. Recovery scans the log, finds the committed transaction, and redoes the updates. If the crash

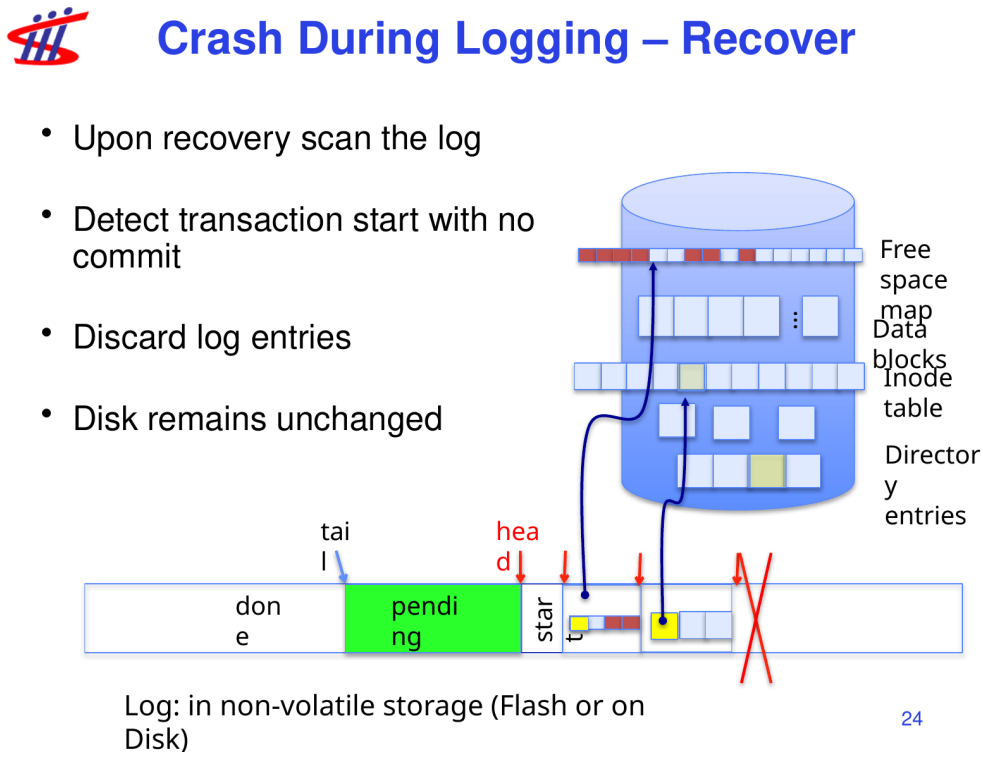


Figure 12: If a crash happens before the commit record is present, recovery discards the incomplete transaction's log records. The transaction is considered not to have happened.

happens again during redo, the next recovery can redo again. This is safe because writing the same final value to the same block multiple times has the same result as writing it once.

Algorithm 2: Redo-Log Recovery After a Crash

Input: Persistent log containing start, update, and commit records
Output: Recovered storage state reflecting all committed transactions

Scan the log from the last checkpoint or from the beginning;
Let C be the set of transaction identifiers with durable commit records;
foreach *log record* `UPDATE`(T, a, v) *in log order* **do**
    if $T \in C$ **then**
        Write value v to home location a;
Discard or ignore update records for transactions with no commit record;
return recovered storage state;

5.5 Journaling File Systems

A journaling file system applies the same principle to file-system metadata. Its goal is that a crash leaves the file system either before or after each logical metadata update, not in an arbitrary middle state.

The cost is write amplification. If every data block is written once to the log and once to its final location, write traffic roughly doubles. Many modern file systems therefore journal metadata only: they log changes to allocation maps, inodes, and directories, while writing file contents directly. This protects structural consistency but may not guarantee that user file contents after a crash contain the newest data unless stronger modes are used.

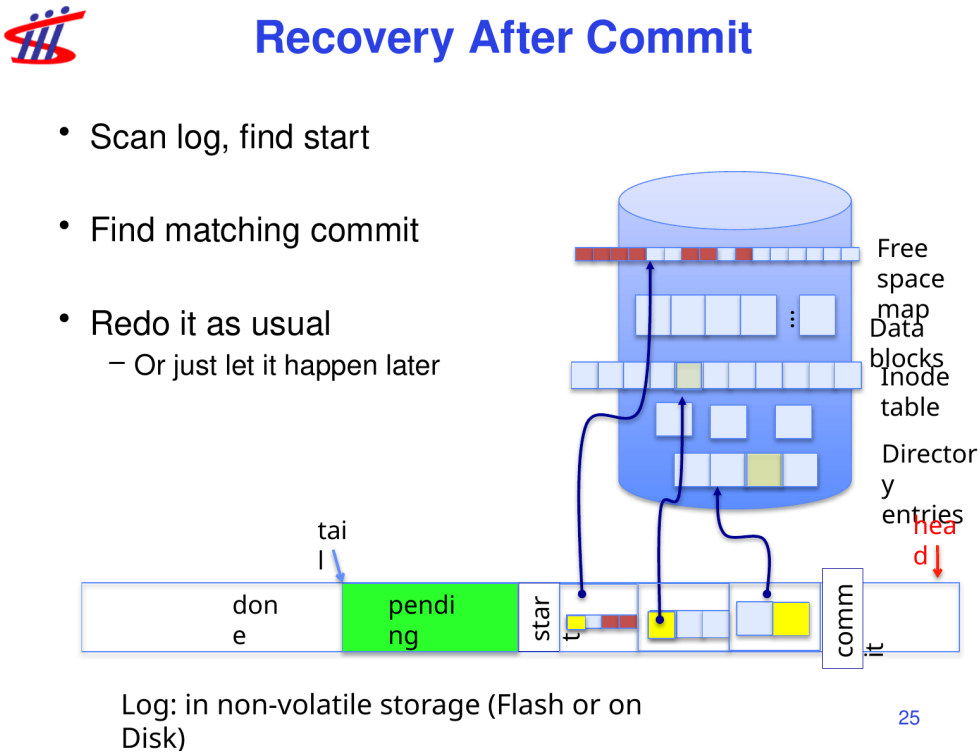


Figure 13: If a crash happens after commit, recovery must ensure that the committed updates eventually reach their home locations. Redo may be repeated safely.

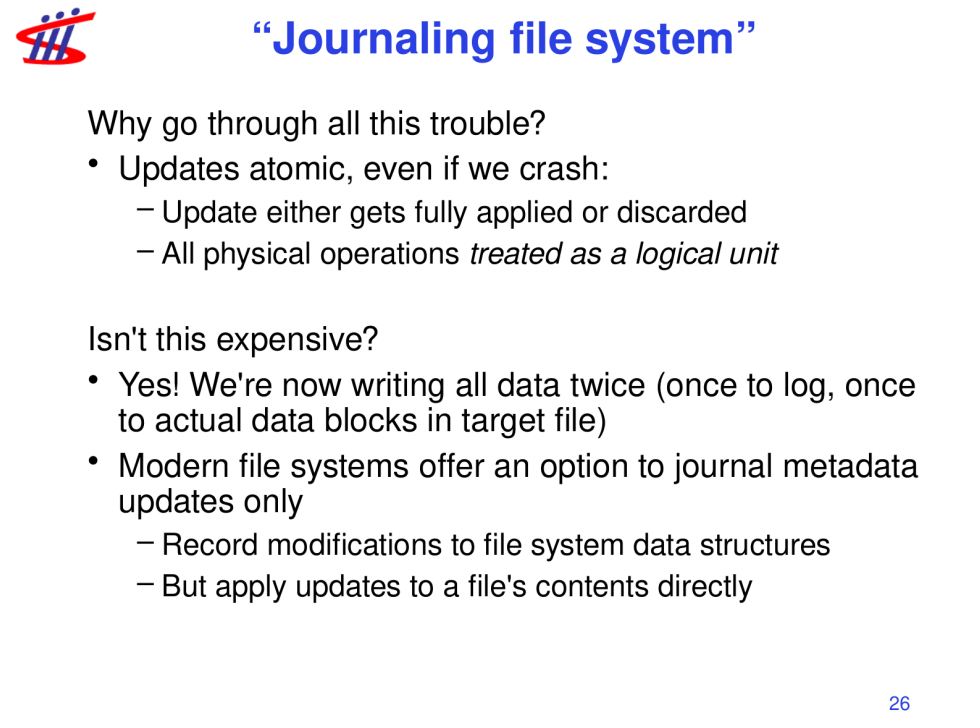


Figure 14: A journaling file system treats several physical writes as one logical unit. Metadata journaling improves crash recovery but can double-write data and therefore has performance costs.

6 Isolation and Concurrent Transaction Execution

6.1 From Multiprogramming to Transactions

Transactions are atomic, but systems do not run them one by one. Disk I/O is slow, users are many, and long transactions may take substantial time. Running all transactions serially would waste CPU time and make interactive users wait unnecessarily. The DBMS therefore interleaves operations from different transactions while preserving a correctness criterion.

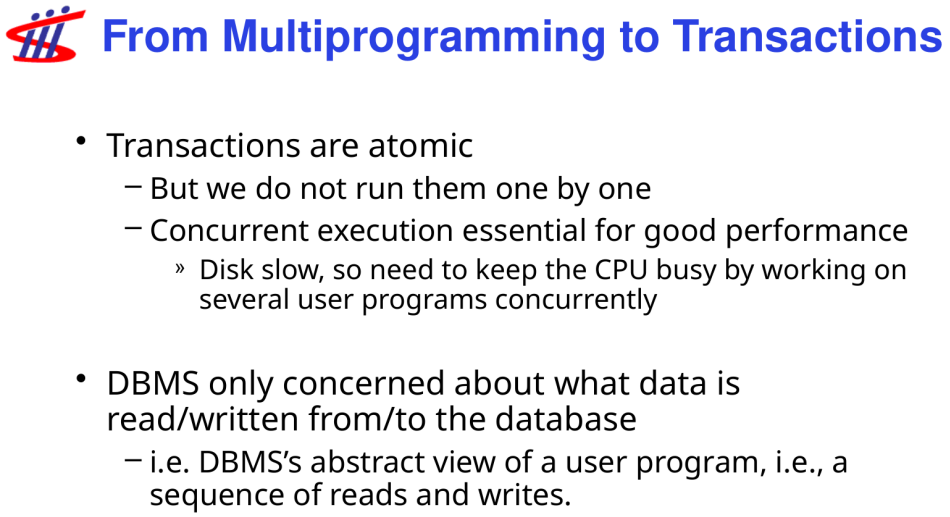


Figure 15: The DBMS abstracts a user program as a sequence of database reads and writes. Internal computation matters only insofar as it determines later reads and writes.

For concurrency control, the DBMS focuses on objects read from and written to the database. Arithmetic such as `A = A - 100` occurs in memory, but the transaction abstraction records the persistent operations: read account A, write account A, read account B, write account B.

```
T1:  R(A); A := A - 100; W(A);
     R(B); B := B + 100; W(B);

T2:  R(B); B := B - 50;  W(B);
     R(A); A := A + 50;  W(A);
```

Listing 5: A money-transfer transaction as database reads and writes.

6.2 Database Locks Are Expensive

A database lock is a higher-level construct than a short in-memory spin lock or mutex. Locks may be handled by a lock manager and recorded in shared tables. In many systems, lock and transaction state interacts with logging and recovery. The critical section may include disk I/O and can last a long time. A transaction that computes and pays salaries for a large organization may touch many records and run for minutes or hours. Locking the entire database for such a transaction would block unrelated work.



Database Locks

- A “lock” in a DB is a higher-level construct than in an OS.
- Think of it as a Readers-Writers Monitor. Why?
- Locks are handled by a lock manager process, and involve updates to a lock table accessible to all clients.
- Setting and releasing locks is a relatively expensive process.

30

Figure 16: Database locks are higher-level and more expensive than ordinary memory locks because they are managed globally and interact with persistent transaction state.

Granularity is therefore a central design choice. Locking an entire database is simple but prevents concurrency. Locking rows gives high concurrency but increases lock-management overhead and can require many lock table entries. Table-level locks are an intermediate compromise. Real systems may also use page locks, key-range locks, intention locks, predicate locks, or multi-version techniques.

7 Transaction Scheduling and Anomalies

7.1 The Goal of Transaction Scheduling

Transaction scheduling has two goals that pull against each other:

- maximize system utilization by interleaving operations from different transactions;
- preserve transaction semantics by ensuring the interleaved execution is equivalent to a correct serial execution.

A *serial schedule* runs one transaction completely before another. If each transaction is correct by itself, a serial schedule is accepted as correct. A concurrent schedule is *serializable* if its result is equivalent to some serial schedule. It does not need to match a particular real-time order unless the system promises stricter semantics; it must match some order.

Definition

A **serializable schedule** is an interleaving of operations whose effect on the database and transaction outputs is identical to the effect of executing the same transactions one at a time in some serial order.

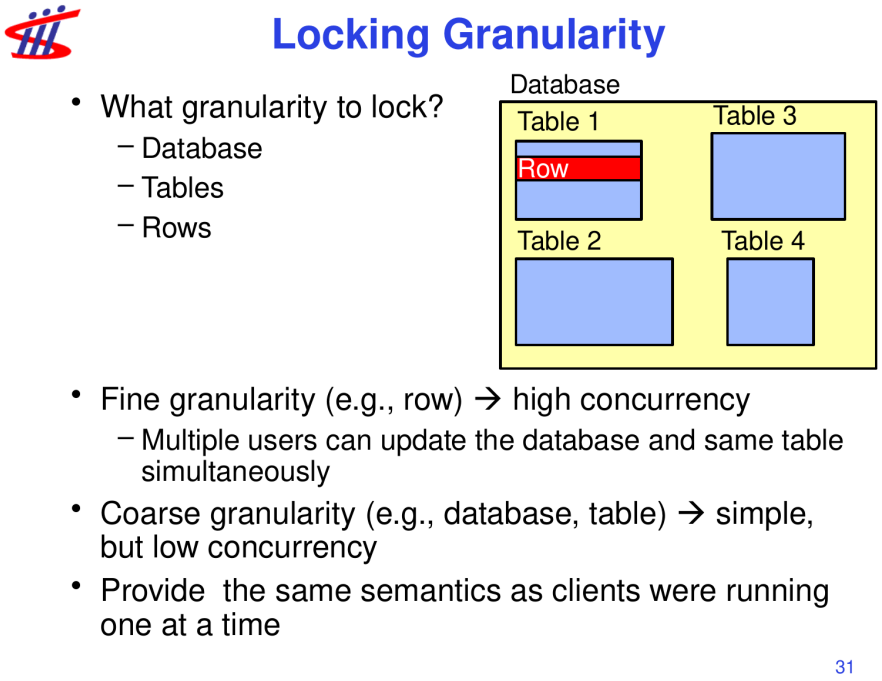


Figure 17: Lock granularity trades simplicity against concurrency. Coarse locks are easy to manage but block more transactions; fine-grained locks allow more interleaving but increase overhead.

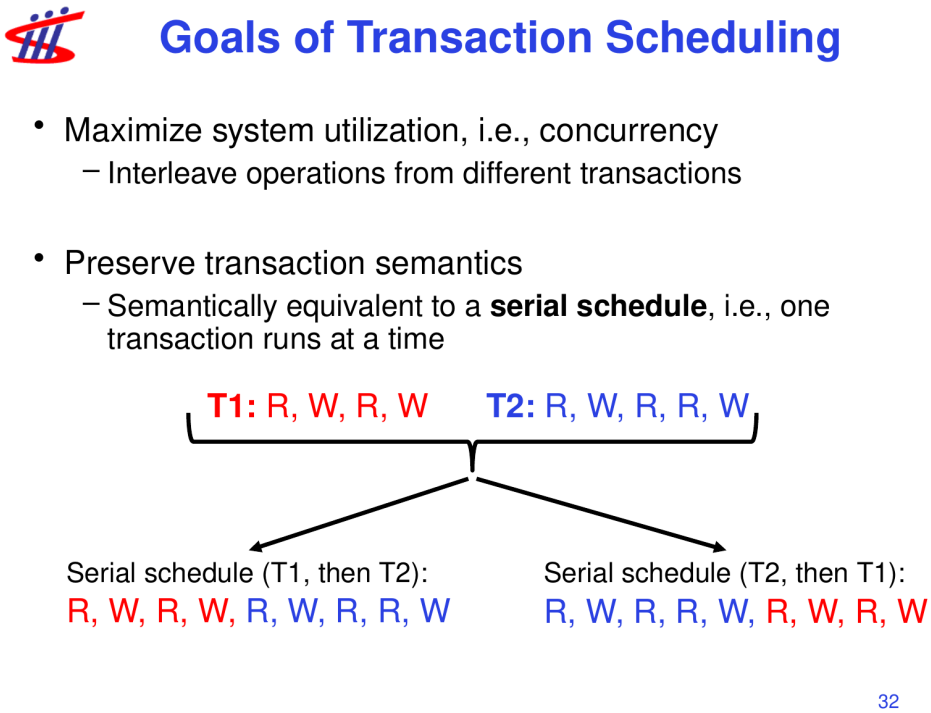


Figure 18: Transaction scheduling seeks concurrency without changing the logical result. A serial schedule is obviously correct but may be unnecessarily slow.

7.2 Why Anomalies Always Involve Writes

If two transactions only read the same object, they do not change persistent state and cannot create an ordering conflict by themselves. Anomalies arise when at least one operation writes: a later write may overwrite a previous write, a write may change a value after another transaction read it, or a transaction may read a value that another transaction later aborts or overwrites.

7.3 Write-Write Conflict

A write-write conflict can produce a final state that is not equivalent to any serial order. Suppose T_1 writes both A and B, and T_2 also writes both A and B. If the final value of A comes from T_2 while the final value of B comes from T_1, then neither T_1-then-T_2 nor T_2-then-T_1 explains the result.

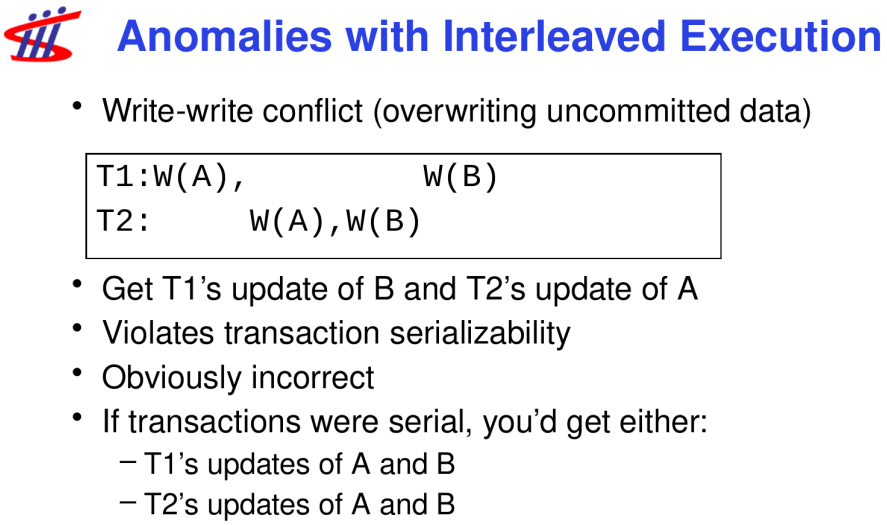


Figure 19: A write-write conflict can mix the effects of two transactions. The final state may contain A from one transaction and B from another, which no serial order would produce.

This is analogous to two users editing the same file in incompatible ways: one user's changes overwrite one part, another user's changes overwrite another part, and the final file does not represent either complete version.

7.4 Read-Write Conflict and Unrepeatable Reads

A read-write conflict occurs when a transaction reads a value, bases a decision on it, and another transaction changes that value before the first transaction commits. If the first transaction reads the same object again, it may observe a different value; this is an unrepeatable read.

The e-commerce example makes the tradeoff vivid. A user sees a discounted television in stock, hesitates, and later tries to buy it, but another user has already purchased the last unit. If "view inventory" and "decide to buy" were one strong transaction, the inventory would need to be protected while the user thinks. Real shopping systems typically do not do this because users may view pages and disappear, leaving inventory locked. Instead, they weaken the semantics: seeing a product in stock is not a reservation unless the application explicitly creates one.

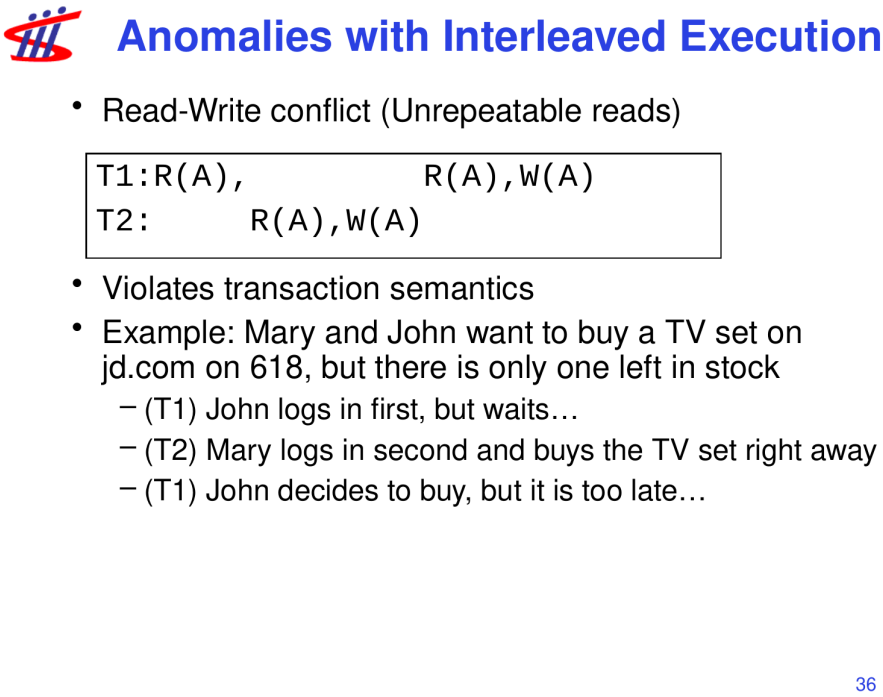


Figure 20: An unrepeatable read occurs when a transaction observes a value and then another transaction updates that value before the first transaction finishes.

7.5 Write-Read Conflict and Dirty Reads

A write-read conflict can expose uncommitted intermediate state. Transaction T_1 writes a tentative value of A; transaction T_2 reads it; then T_1 overwrites A again or aborts. Transaction T_2 has made decisions based on a value that never became a committed database fact.

Dirty reads are rarely useful. During a transaction, the DBMS or application may try intermediate values, reorder computations, update temporary structures, or eventually abort. External transactions should not depend on such states. Preventing dirty reads is one reason many systems hold write locks until commit in stricter locking variants.

8 Serializability and Conflict Serializability

8.1 Equivalent and Serializable Schedules

Two schedules are equivalent if, for any initial database state, they produce the same final database state and transaction outputs. A schedule is serializable if it is equivalent to some serial schedule. This definition is semantic: it depends on the meaning of operations and the data. Semantic serializability is the ideal but can be difficult to decide in general.

8.2 Conflicting Operations

A syntactic approximation is *conflict serializability*. Two operations conflict if:

1. they belong to different transactions;
2. they access the same data object;
3. at least one is a write.

Two schedules are conflict equivalent if they contain the same operations and every pair of conflicting operations appears in the same order. A schedule is conflict serializable if it is conflict



Anomalies with Interleaved Execution

- Write-read conflict (reading uncommitted data)

```
T1:R(A),W(A),      W(A)
T2:           R(A),     …
```

- Example:
  - (T1) A user updates value of A in two steps
  - (T2) Another user reads the intermediate value of A, which can be inconsistent
  - Violates transaction semantics since T2 is not supposed to see intermediate state of T1

37

Figure 21: A dirty read occurs when one transaction reads an intermediate value written by another transaction before that other transaction commits.



Transaction Scheduling

- **Serial schedule:** A schedule that does not interleave the operations of different transactions
  - Transactions run serially (one at a time)
- **Equivalent schedules:** For any storage/database state, the effect (on storage/database) and output of executing the first schedule is identical to the effect of executing the second schedule
- **Serializable schedule:** A schedule that is equivalent to *some* serial execution of the transactions
  - Intuitively: with a serializable schedule you only see things that could happen in situations where you were running transactions one-at-a-time

38

Figure 22: Transaction scheduling defines serial schedules, equivalent schedules, and serializable schedules. Serializable executions preserve the illusion that transactions ran one at a time.

equivalent to some serial schedule.

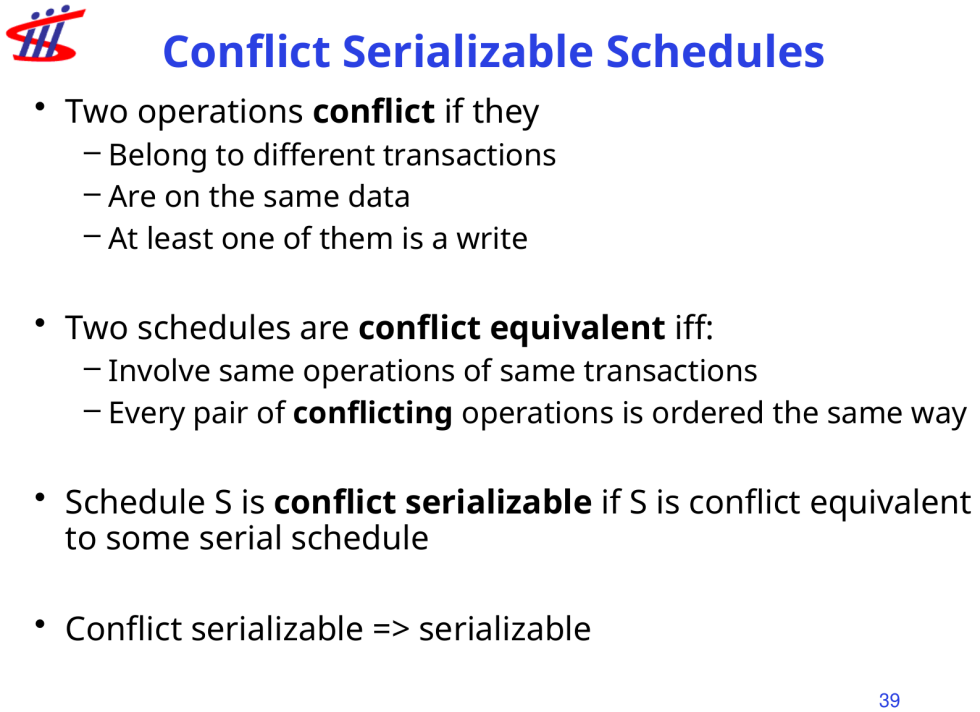


Figure 23: Conflict serializability is a syntactic, efficiently checkable sufficient condition for serializability. It preserves the order of all conflicting operation pairs.

The key intuition is swapping. If two adjacent operations from different transactions do not conflict, swapping them cannot change the result. If repeated swaps of non-conflicting operations can transform an interleaved schedule into a serial one, then the original schedule is conflict serializable.

8.3 Dependency Graphs

A dependency graph, also called a precedence graph, gives an efficient test for conflict serializability. Each node is a transaction. Add an edge $T_i \to T_j$ if an operation of T_i conflicts with an operation of T_j and the T_i operation appears earlier in the schedule.

Theorem

A schedule is conflict serializable if and only if its dependency graph is acyclic. If the graph is acyclic, any topological ordering of the graph gives a serial order conflict-equivalent to the schedule.



Conflict Equivalence – Intuition

- If you can transform an interleaved schedule by swapping *consecutive non-conflicting* operations of *different transactions* into a serial schedule, then the original schedule is **conflict serializable**
- Example:

```
T1:R(A),W(A),          R(B),W(B)
T2:          R(A),W(A),          R(B),W(B)
```

⇩

```
T1:R(A),W(A),     R(B),     W(B)
T2:          R(A),     W(A),     R(B),W(B)
```

⇩

```
T1:R(A),W(A),R(B),          W(B)
T2:               R(A),W(A),     R(B),W(B)
```

40

Figure 24: Non-conflicting adjacent operations may be swapped. Repeated safe swaps can move operations until the schedule becomes serial.



Dependency Graph

- **Dependency graph:**
  - Transactions represented as nodes
  - Edge from Ti to Tj:
    - » an operation of Ti conflicts with an operation of Tj
    - » Ti appears earlier than Tj in the schedule
- **Theorem:** Schedule is conflict serializable if and only if its dependency graph is acyclic

43

Figure 25: The dependency graph contains one node per transaction and one directed edge for each conflict ordering between transactions. Acyclicity is equivalent to conflict serializability.

Algorithm 3: Testing Conflict Serializability by a Dependency Graph

Input: Schedule S consisting of read and write operations from transactions $T_1, \ldots, T_n$
Output: Whether S is conflict serializable and, if so, one serial order

```
Create a directed graph G with one node for each transaction;
foreach ordered pair of operations (o_i, o_j) in S with o_i before o_j do
    if o_i and o_j belong to different transactions, access the same object, and at least one is a write then
        Add edge T(o_i) → T(o_j) to G;
if G has a directed cycle then
    return not conflict serializable;
Compute a topological ordering of G;
return conflict serializable with that serial order;
```

8.4 Examples and Limits

If all conflicts point from T_1 to T_2, the dependency graph has no cycle. The schedule is equivalent to T_1 followed by T_2.

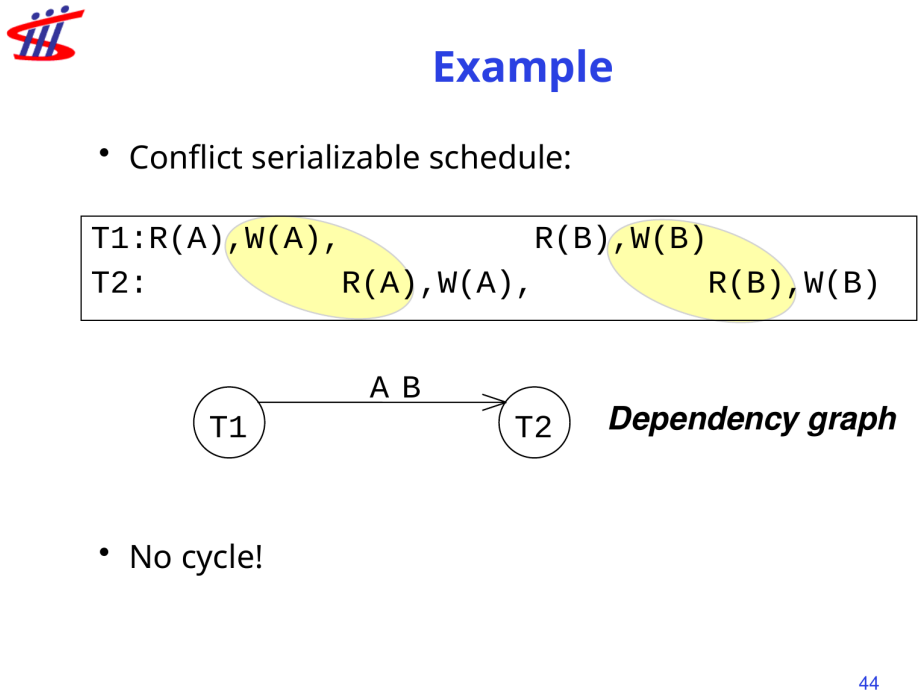


Figure 26: A conflict-serializable schedule yields a dependency graph with no cycle. A topological order gives the equivalent serial execution.

If some conflicts force T_1 before T_2 and others force T_2 before T_1, the dependency graph contains a cycle. No serial order can satisfy both requirements.

Conflict serializability is stronger than serializability. Every conflict-serializable schedule is serializable, but not every serializable schedule is conflict serializable. For example, if a final write overwrites all earlier effects, the final database state may be equivalent to a serial execution even though the conflict graph has a cycle. Conflict serializability rejects such cases because it does not reason about semantics; it only reasons about operation order.

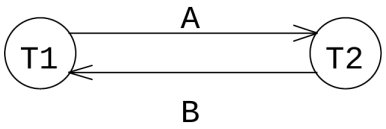
This conservatism is intentional. General serializability depends on transaction semantics and is hard to decide. Conflict serializability is efficient, mechanical, and enforceable by simple



Example

- Conflict that is *not* serializable:

```
T1:R(A),W(A),                    R(B),W(B)
T2:           R(A),W(A),R(B),W(B)
```



Dependency graph

- Cycle: The output of T1 depends on T2, and vice-versa

45

Figure 27: A cyclic dependency graph indicates contradictory conflict orderings: T_1 must precede T_2 and T_2 must precede T_1.

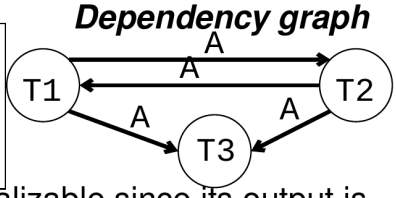


Serializability ≠> Conflict Serializability

- Following schedule is **not** conflict serializable

```
T1:R(A),      W(A),
T2:      W(A),
T3:                 W(A)
```

Dependency graph



- However, the schedule is serializable since its output is equivalent with the following serial schedule

```
T1:R(A),W(A),
T2:           W(A),
T3:                W(A)
```

- Note: deciding whether a schedule is serializable (not conflict-serializable) is NP-complete

48

Figure 28: Conflict serializability is conservative. A schedule can be serializable by semantic outcome while not being conflict serializable by syntactic conflict order.

protocols. The cost is that it may reject some safe schedules and thereby sacrifice concurrency.

9 Locks and Two-Phase Locking

9.1 Shared and Exclusive Locks

Databases often use shared and exclusive locks. A shared lock, or S lock, allows multiple transactions to read the same data concurrently. An exclusive lock, or X lock, is required for writing and excludes other readers and writers under the simple compatibility matrix.

	S requested	**X requested**
S held	compatible	incompatible
X held	incompatible	incompatible

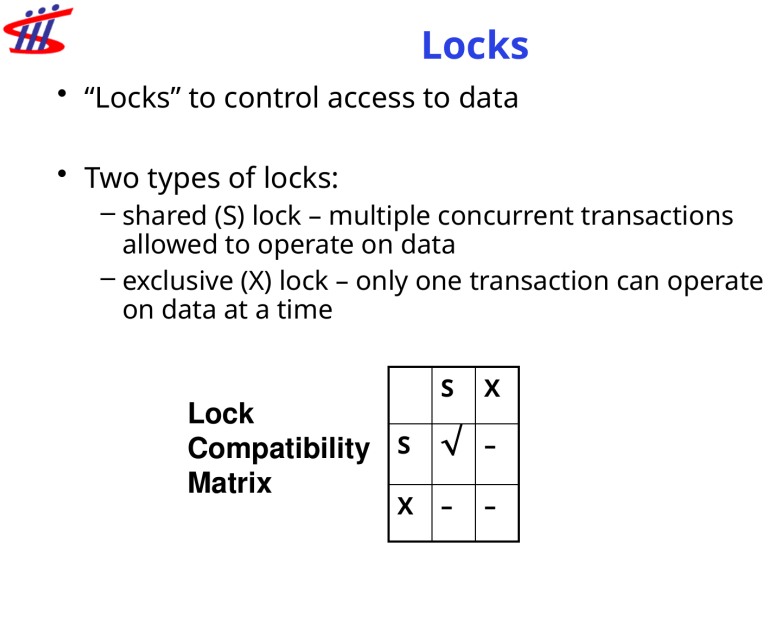


Figure 29: The simple shared/exclusive compatibility matrix captures reader-writer locking. Multiple readers can coexist; any writer excludes other accesses.

A transaction must obtain a suitable lock before accessing an object: an S lock before reading, an X lock before writing. If a transaction first reads and later writes the same object, it may upgrade an S lock to an X lock. In the two-phase-locking view, such an upgrade counts as acquiring additional lock strength; downgrading or releasing moves in the opposite direction.

9.2 Why Immediate Unlocking Is Not Enough

A tempting rule is: lock before each operation, unlock immediately after. This prevents simultaneous conflicting access at a single instant, but it does not guarantee serializability. A transaction might read A and release its S lock; another transaction then writes A; later the first transaction accesses B in a way that creates a cycle. Correctness depends on the ordering of the whole transaction, not only on each individual read or write.

The most conservative correct rule is to hold all locks until commit. This is simple and safe, but it may reduce concurrency severely. Two-phase locking is a more flexible rule that still guarantees conflict serializability.

9.3 Two-Phase Locking

Two-phase locking (2PL) imposes two phases on each transaction:

- **Growing phase:** the transaction may acquire locks but may not release locks.
- **Shrinking phase:** once the transaction releases any lock, it may not acquire any new lock.

The instant at which the transaction first releases a lock is called its *lock point*.

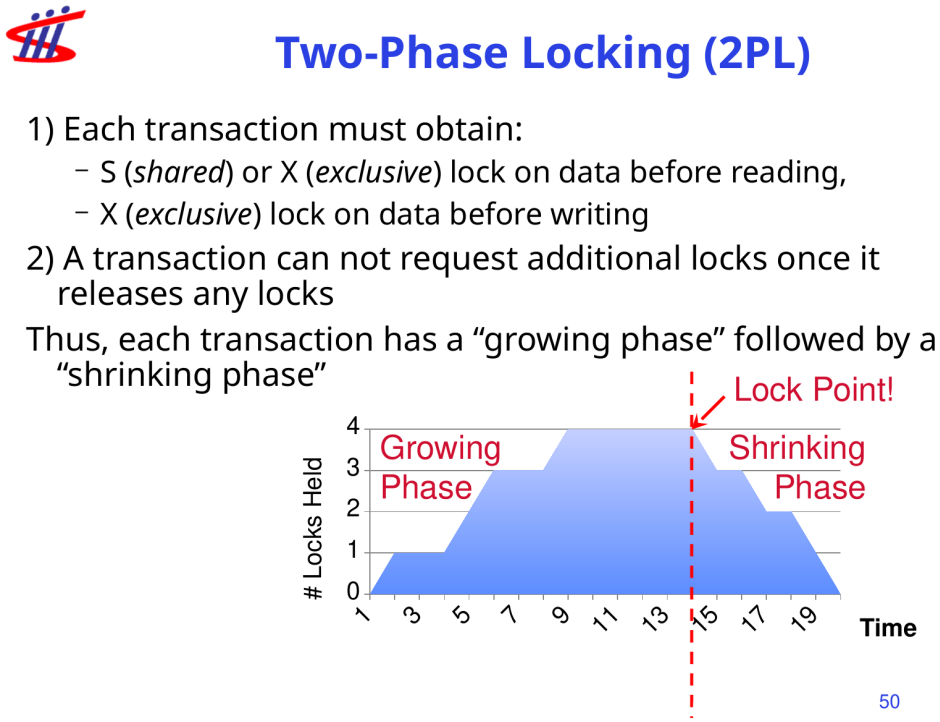


Figure 30: Two-phase locking divides execution into a growing phase and a shrinking phase. After the first release, the transaction may not acquire new locks.

The intuition is that after the lock point, every conflict with other transactions is forced to occur in a consistent direction. Before the transaction releases locks, conflicting transactions must wait. After release begins, the transaction will not acquire new locks that could create a reverse dependency.

Deep Dive

Two-phase locking does not require a transaction to know all future locks at the beginning. In the worst case, it can simply keep locks until the end. If the DBMS has a query plan and can know that certain data will no longer be accessed, it may release locks earlier, but only after it is sure no new locks will be needed. Early release is an optimization, not the fundamental correctness rule.

9.4 Why 2PL Guarantees Conflict Serializability

A schedule produced by 2PL has an acyclic dependency graph. Suppose, for contradiction, that the dependency graph has a cycle. An edge $T_i \to T_j$ means that T_i performed a conflicting operation before T_j, so T_i must have acquired the relevant lock before T_j could proceed. Around a cycle, each transaction would need to wait for another in a circular manner. Such a cycle would become a deadlock rather than a completed schedule. A completed 2PL schedule therefore cannot contain a dependency cycle.

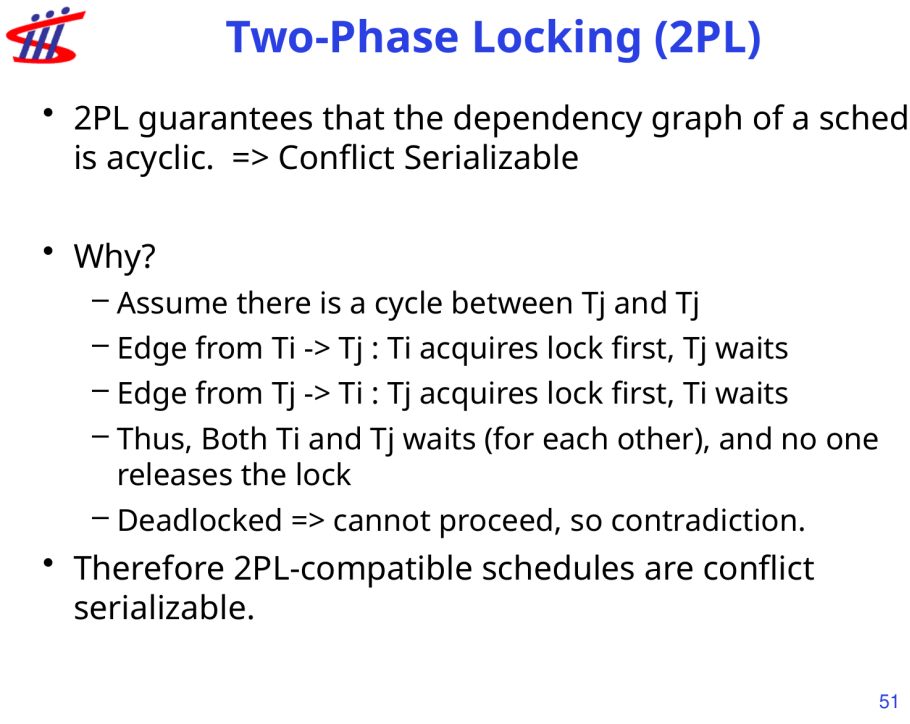


Figure 31: Two-phase locking guarantees conflict serializability because a completed cyclic dependency would imply circular waiting. The system would deadlock instead of completing such a schedule.

Deadlocks are still possible under locking. The transaction system resolves them by aborting and rolling back one or more transactions, using the recovery machinery already discussed. This is one reason durability and recovery mechanisms are tightly connected to concurrency control: before commit, a transaction can be rolled back and made as if it never happened.

9.5 Strict 2PL and Optimistic Concurrency Control

A common practical variant is strict two-phase locking. Strict 2PL holds exclusive locks, and often all locks, until commit or abort. This reduces concurrency but prevents other transactions from reading data written by an uncommitted transaction, simplifying recovery and avoiding cascading aborts.

At the other end of the spectrum is optimistic concurrency control. Instead of acquiring locks early, transactions execute speculatively while the system records their read and write sets. At validation time, the system checks whether a dangerous conflict occurred. If so, one transaction aborts and retries. This approach is effective when conflicts are rare, such as workloads where users mostly update independent records. It avoids much locking overhead but can suffer from repeated aborts or livelock under high contention.

Deep Dive

The ability to abort before commit is the source of much flexibility in database systems. A system can allow speculative work, detect that the interleaving was unsafe, and roll back one transaction. Without a recovery mechanism, optimistic concurrency would be dangerous; with logging and undo/redo machinery, it becomes a practical performance technique.

10 Putting the Lecture Together

Lecture 12 connects three levels of abstraction:

1. **Database abstraction:** relations, schemas, queries, and query plans let applications operate on structured persistent data rather than raw files.
2. **Recovery abstraction:** transactions and logs turn many physical writes into one logical, crash-resilient update.
3. **Concurrency abstraction:** serializability, conflict serializability, dependency graphs, and 2PL allow interleaving while preserving the illusion of one-at-a-time execution.

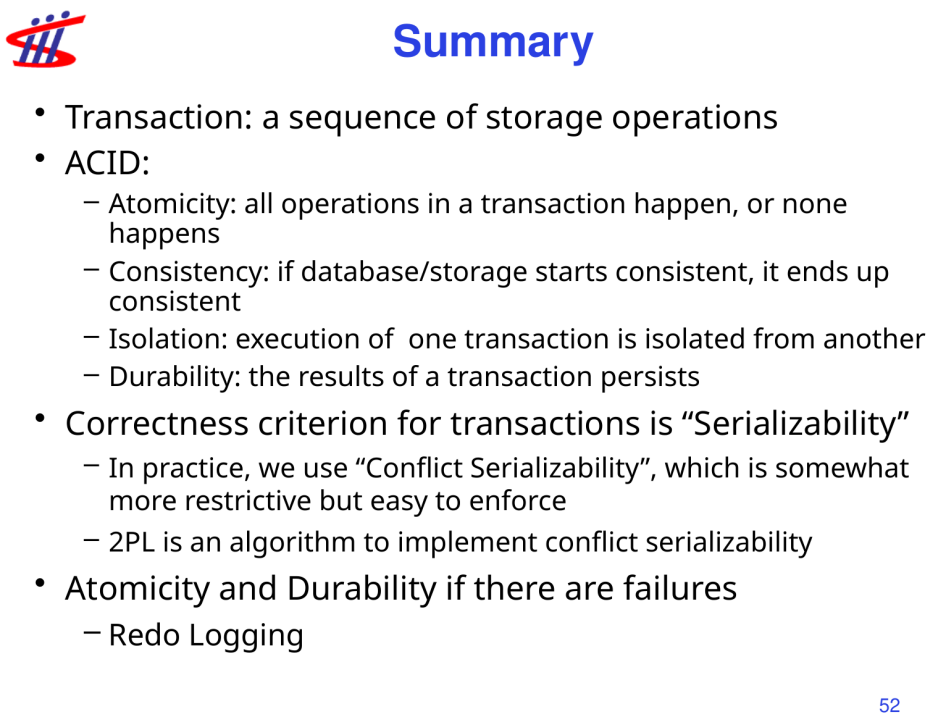


Figure 32: Lecture summary: transactions provide ACID properties; redo logging supports atomicity and durability; conflict serializability and 2PL support isolation.

The most important distinction is between what is desirable and what is enforceable. Serializability is the semantic correctness criterion, but conflict serializability is the syntactic criterion that systems can efficiently test and enforce. Atomicity and durability are desired properties, but redo logging is the concrete technique that provides them. Isolation is desired, but 2PL is one practical protocol that provides a strong version of it.

11 Summary

A database system extends operating-system ideas into the world of persistent structured data. Schemas provide meaning to bytes; queries allow declarative access; query plans let the system choose efficient execution. A transaction is a sequence of storage actions treated as one logical unit. ACID summarizes the transaction contract: atomicity, consistency, isolation, and durability.

Persistent storage makes atomicity difficult because disk writes are not inherently atomic and multi-block updates can crash halfway through. Write-ahead logging and redo recovery solve this by recording intended updates in nonvolatile storage before installing them. A commit record

separates incomplete transactions, which may be discarded, from committed transactions, which must be replayed until durable.

Concurrency makes isolation difficult because interleaved transactions can create write-write conflicts, unrepeatable reads, dirty reads, and other anomalies. Serializability says that a concurrent execution is correct if it is equivalent to some serial order. Conflict serializability gives a conservative, efficiently enforceable sufficient condition based on conflicting operations. A dependency graph is acyclic exactly when a schedule is conflict serializable. Two-phase locking enforces conflict serializability by requiring each transaction to acquire locks before its lock point and release locks only after it stops acquiring new ones.

The unifying lesson is that systems design repeatedly replaces fragile physical reality with disciplined abstractions. Logs turn crash-prone partial writes into all-or-nothing updates. Locking protocols turn unpredictable interleavings into serializable schedules. Schemas and queries turn raw disk blocks into meaningful persistent data.

12 Quizzes & Exercises

Conceptual Questions

1. Explain the difference between a data model and a schema. Why is a foreign key not the same as a memory pointer?
2. In the relational model, why is `WHERE GPA > 3` a selection, while `SELECT sid, name, GPA` is a projection?
3. What are the four ACID properties? Give one concrete failure or concurrency example that violates each property.
4. Why is creating a file not naturally atomic on persistent storage? Name at least four pieces of state that may be modified during file creation.
5. What is the role of the commit record in a redo log? What should recovery do if it finds a start record and update records but no commit record?
6. Why can redo recovery safely repeat an update? Give an example of an idempotent storage operation.
7. Why is locking an entire database usually too conservative? What are the tradeoffs between database-level, table-level, and row-level locks?
8. Define a serial schedule and a serializable schedule. Why do database systems not simply run every transaction serially?
9. What three conditions make two operations conflict in conflict-serializability theory?
10. Why is conflict serializability conservative compared with general serializability?
11. How does a dependency graph test conflict serializability? What does a cycle mean?
12. State the two rules of two-phase locking. What is the lock point?
13. Why does simply locking before each read or write and unlocking immediately afterward fail to guarantee serializability?
14. How can two-phase locking cause deadlock? Why can a transaction system recover from deadlock by aborting a transaction?
15. When might optimistic concurrency control outperform locking? When might it perform poorly?

Practice Problems

Problem 1: Query operators. For the schema `Students(sid,name,gpa)` and `Enrolled(sid,cid,grade)`, write a SQL query that returns the names of students enrolled in course `'CS101'` with GPA at least 3.5. Identify the selection, projection, and join operations.

Problem 2: Redo log recovery. A log contains the following records:

`START(T1)`, `UPDATE(T1,A,10)`, `START(T2)`, `UPDATE(T2,B,20)`, `COMMIT(T1)`.

The system crashes before any home-location writes are known to be complete. Which updates must recovery redo? Which updates must it ignore?

Problem 3: Conflict graph. For the schedule

$$S = R_1(A),\ W_2(A),\ R_1(B),\ W_1(B),\ R_2(B),\ W_2(B),$$

construct the dependency graph and decide whether the schedule is conflict serializable.

Problem 4: Non-conflicting swaps. Show a sequence of swaps that transforms the schedule

$$R_1(A),\ W_1(A),\ R_2(B),\ W_2(B),\ R_1(C),\ W_1(C)$$

into a serial schedule. Explain why each swap is safe.

Problem 5: Two-phase locking. A transaction obtains locks in this order: `S(A)`, `X(B)`, releases `A`, obtains `S(C)`, releases `B`, releases `C`. Does it obey 2PL? Explain.

Answer Sketches

Problem 1. One possible query is:

```
SELECT S.name
FROM Students S, Enrolled E
WHERE S.sid = E.sid
  AND E.cid = 'CS101'
  AND S.gpa >= 3.5;
```

The join condition is `S.sid = E.sid`; the selections are `E.cid = 'CS101'` and `S.gpa >= 3.5`; the projection is `S.name`.

Problem 2. Recovery must redo `UPDATE(T1,A,10)` because `T1` has a commit record. It must ignore `UPDATE(T2,B,20)` because `T2` has no commit record.

Problem 3. On object A, $R_1(A)$ precedes $W_2(A)$, so add $T_1 \to T_2$. On object B, $W_1(B)$ precedes $R_2(B)$ and $W_2(B)$, so again add $T_1 \to T_2$. There is no reverse edge, so the graph is acyclic and the schedule is conflict serializable, equivalent to T_1 before T_2.

Problem 4. The T_2 operations on B do not conflict with the T_1 operations on A or C, since they access different objects. Move $R_2(B), W_2(B)$ after $W_1(C)$ by swapping them rightward across $R_1(C), W_1(C)$, producing T_1 followed by T_2.

Problem 5. It violates 2PL. Once the transaction releases `A`, it has entered the shrinking phase and may not acquire `S(C)` afterward.